



UNIVERSITÀ  
POLITECNICA  
DELLE MARCHE

FACOLTÀ DI INGEGNERIA

CORSO DI LAUREA IN INGEGNERIA INFORMATICA E DELL'AUTOMAZIONE

---

# VisualSync: Multi-Camera Synchronization

Autori:

**Davide Acciarri**  
**Giacomo Marcucci**

Tutor:

**Rocco Pietrini**

Anno Accademico 2025-2026



# Indice

|          |                                                                        |           |
|----------|------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione</b>                                                    | <b>1</b>  |
| <b>2</b> | <b>Stato dell'arte</b>                                                 | <b>3</b>  |
| 2.1      | Tecniche basate sull'apprendimento: Sync-NeRF . . . . .                | 3         |
| 2.2      | Tecniche Structure-from-Motion (SfM) . . . . .                         | 3         |
| 2.3      | Tecniche basate sul tracking temporale e sulle corrispondenze spaziali | 4         |
| 2.4      | Tecniche basate su modelli di fondazione: Uni4D . . . . .              | 4         |
| <b>3</b> | <b>Strumenti e architetture</b>                                        | <b>5</b>  |
| 3.1      | Struttura Dataset . . . . .                                            | 5         |
| 3.2      | Modelli . . . . .                                                      | 6         |
| 3.2.1    | Grounded-SAM 2 . . . . .                                               | 6         |
| 3.2.2    | DEVA . . . . .                                                         | 9         |
| 3.2.3    | VGGT . . . . .                                                         | 10        |
| 3.2.4    | CoTracker3 . . . . .                                                   | 11        |
| 3.2.5    | MASt3R . . . . .                                                       | 12        |
| 3.3      | Le tre fasi di visualsync . . . . .                                    | 13        |
| 3.3.1    | Fase 0: Estrazione degli indizi visivi . . . . .                       | 14        |
| 3.3.2    | Fase 1: Stima degli offset a coppie . . . . .                          | 15        |
| 3.3.3    | Fase 2: Stima globale degli offset . . . . .                           | 16        |
| <b>4</b> | <b>Miglioramenti e ottimizzazioni architetturali</b>                   | <b>17</b> |
| <b>5</b> | <b>Risultati finali</b>                                                | <b>21</b> |
| 5.1      | Iperparametrizzazione dell'offset range . . . . .                      | 21        |
| 5.2      | Valutazioni per i diversi FPS . . . . .                                | 22        |
| 5.3      | Risultati generali . . . . .                                           | 23        |
| 5.4      | Risultati CodeCarbon . . . . .                                         | 24        |
| <b>6</b> | <b>Conclusioni</b>                                                     | <b>25</b> |
| <b>7</b> | <b>Possibili sviluppi futuri</b>                                       | <b>27</b> |



## Elenco delle figure

|     |                                                       |   |
|-----|-------------------------------------------------------|---|
| 3.1 | Struttura del dataset . . . . .                       | 5 |
| 3.2 | Struttura del dataset dopo il preprocessing . . . . . | 6 |



## **Elenco delle tabelle**





# Capitolo 1

## Introduzione

Con la diffusione di smartphone e dispositivi portatili, è comune che uno stesso evento dinamico, come una partita di calcio, un concerto o una festa in famiglia, venga ripreso da più angolazioni da diverse persone. Tuttavia, allineare questi flussi video per scopi di ricostruzione 4D o editing professionale rimane una sfida complessa a causa della natura frammentaria e casuale delle riprese. Per la sincronizzazione temporale di video multi-camera acquisiti in modo indipendente e non coordinato, è stato progettato un framework di ottimizzazione, VisualSync. I metodi di sincronizzazione esistenti si basano su ambienti controllati, annotazioni manuali, pattern specifici come human pose, segnali audio, o costosi setup hardware come dispositivi time-coded, nessuno dei quali è disponibile in video registrati casualmente. Un altro aspetto da dover affrontare è che i video sono spesso *unposed*, ovvero non si hanno informazioni a priori riguardanti la posizione spaziale, l'orientamento o i parametri intrinseci delle telecamere al momento della ripresa. Inoltre, costruire un sistema pratico e robusto che funzioni su video reali è impegnativo, in quanto gli oggetti dinamici sono a priori sconosciuti e generalmente piccoli e sfocati, e non tutte le viste possono avere sovrapposizione. Il problema parte da un insieme di  $N$  video asincroni che riprendono la stessa scena dinamica da diversi punti di vista e l'obiettivo è sincronizzarli e recuperare un timestamp allineato globalmente. Formalmente, si cerca di stimare un offset temporale per ogni video da applicare al suo tempo di clock originale fuori sincronizzazione. Dopo la sincronizzazione, i frame che condividono lo stesso tempo di clock corrisponderanno allo stesso momento in tutti i video. È possibile vedere la sincronizzazione globale come un problema di minimizzazione dell'energia, ovvero, miriamo a trovare gli offset che allineino meglio tutte le coppie di video, minimizzando il loro errore di sincronizzazione *pairwise*. La minimizzazione dell'energia pone tre sfide: il problema di ottimizzazione è altamente non convesso, la formulazione è a tempo continuo e le osservazioni arrivano a tempi di frame discreti, infine, negli scenari reali, è difficile associare traiettorie dense tra fotocamere con punti di vista molto diversi e andare a stimare pose accurate per fotocamere in movimento. L'obiettivo del progetto è sviluppare una metodologia per la sincronizzazione temporale di viste acquisite da più telecamere. Come punto di partenza è stato preso in esame il lavoro proposto nel paper *VisualSync: Multi-Camera Synchronization via Cross-View Object Motion*[1], accettato alla conferenza NeurIPS 2025, che propone un approccio basato

## *Capitolo 1 Introduzione*

sull'analisi del moto degli oggetti osservati da prospettive differenti per stimare gli offset temporali tra i flussi video.

## Capitolo 2

### Stato dell'arte

La sincronizzazione e la ricostruzione di scene dinamiche multi-camera hanno subito una trasformazione radicale grazie all'avvento dei modelli di fondazione visiva e di nuove tecniche di ottimizzazione geometrica. La sincronizzazione video è stata esplorata da più prospettive. I metodi basati sulla geometria, stimano offset temporali usando la geometria epipolare, ma tipicamente assumono scene statiche o punti di vista fissi. Gli approcci human-centric usano la *human pose* come segnale di sincronizzazione, ma sono limitati dall'accuratezza della stima della pose, dal numero di persone nella scena, e faticano in scenari senza attività umana prominente. Gli approcci basati sull'audio usano indizi audio per la sincronizzazione, ma funzionano solo in ambienti silenziosi e non sono generalmente applicabili a contesti reali in cui l'audio è rumoroso o non disponibile.

#### 2.1 Tecniche basate sull'apprendimento: Sync-NeRF

I metodi basati sull'apprendimento come Sync-NeRF, ottimizzano congiuntamente pose e offset temporali, ma spesso sono vincolati ad ambienti o tipi di oggetti specifici. VisualSync supera questi limiti sfruttando modelli visivi pre-addestrati e inquadrando la sincronizzazione come un problema di ottimizzazione basato sull'epipolarità. Ragionando congiuntamente su strutture statiche e movimenti dinamici in primo piano, è stata definita una soluzione generalizzabile per allineare video asincroni e non posizionati in scenari reali complessi.

#### 2.2 Tecniche Structure-from-Motion (SfM)

Per la Multi-View Structure-from-Motion ci sono tecniche Structure-from-Motion (SfM), come COLMAP, che hanno significativamente avanzato le pipeline di ricostruzione 3D producendo pose accurate delle fotocamere da immagini e video multi-vista. Modelli più recenti come HLOC, Hierarchical-Localization, e VGGT, Visual Geometry Grounded Transformer, si basano su questi progressi usando feature basate sull'apprendimento, transformer per gestire il matching su larga scala e la stima della pose. Sebbene questi metodi raggiungano forti prestazioni nella stima della geometria delle fotocamere, sono carenti nella sincronizzazione di scene dinamiche, poiché si

basano principalmente su indizi visivi statici. VisualSync decompone esplicitamente la scena in componenti statiche e dinamiche, sfruttando gli indizi statici per la stima della pose e gli indizi dinamici dagli oggetti in movimento per eseguire una robusta sincronizzazione temporale tra viste.

### **2.3 Tecniche basate sul tracking temporale e sulle corrispondenze spaziali**

Per il tracking e le corrispondenze ci sono modelli come CoTracker che tracciano densamente punti nel tempo, offrendo una forte coerenza temporale. Tuttavia, non modellano le corrispondenze spaziali tra punti di vista diversi. Invece, MAST3R, si concentra sul matching spaziale e la ricostruzione stereo, fornendo corrispondenze cross-view dense, ma non gestisce la dinamica temporale, specialmente in scene in movimento. VisualSync colma questa lacuna costruendo corrispondenze cross-view spazio-temporali, integrando sia il tracking temporale che il matching spaziale per abilitare una sincronizzazione accurata in contesti video multi-vista dinamici.

### **2.4 Tecniche basate su modelli di fondazione: Uni4D**

Il framework Uni4D ha come obiettivo quello di unificare diversi modelli di fondazione pre-addestrati per ricostruire scene 4D da un singolo video casuale. Uni4D eccelle nella ricostruzione della geometria dinamica e nella stima della posa della telecamera ma, quando viene utilizzato come baseline per la sincronizzazione multi-video, in presenza di oggetti dinamici, piccoli o distanti, presenta delle incertezze. VisualSync adotta esplicitamente la pipeline di segmentazione degli oggetti dinamici introdotta da Uni4D, infatti utilizza la logica di Uni4D per identificare cosa si muove nella scena e separarlo dallo sfondo statico. Uni4D si basa su Recognize Anything e un filtraggio tramite LLM, invece, VisualSync, utilizza direttamente GPT-4o per identificare le classi dinamiche, mantenendo però la struttura di propagazione temporale tipica di Uni4D. Nel nostro progetto il modello GPT-4o non è stato utilizzato in quanto sostituito da una definizione esplicita delle classi dinamiche tramite un file JSON. Questo rende il processo più veloce e deterministico per dataset di laboratorio dove vengono individuate mani e braccia come classi di interesse. La segmentazione si è spostata dai modelli addestrati su classi specifiche come persone o veicoli, a sistemi universali. Il modello SAM 2, Segment Anything Model 2, è l'attuale punto di riferimento ed è un modello fondamentale per la risoluzione del problema della segmentazione visiva guidata da prompt in immagini e video tramite una memoria di streaming che permette di mantenere la coerenza temporale anche in sequenze lunghe e complesse. Framework come DEVA hanno ulteriormente raffinato questo approccio, introducendo un sistema di propagazione bidirezionale che denormalizza gli errori dei modelli a frame singolo.

# Capitolo 3

## Strumenti e architetture

### 3.1 Struttura Dataset

La struttura del dataset è organizzata in modo gerarchico per gestire le diverse scene di video e i relativi punti di vista. Le diverse scene vengono rappresentate da 18 sottocartelle nominate per ID e all'interno di ogni sottocartella sono presenti tre sottocartelle specifiche per le diverse angolazioni di ripresa:

- vista dall'alto (TOP);
- vista in prima persona (FPV);
- vista in terza persona (TPV).

I video originali sono contenuti in queste sottocartelle, mentre i video sincronizzati che forniscono il Ground Truth da utilizzare successivamente nelle operazioni di confronto, sono collocati nella cartella dell'ID.

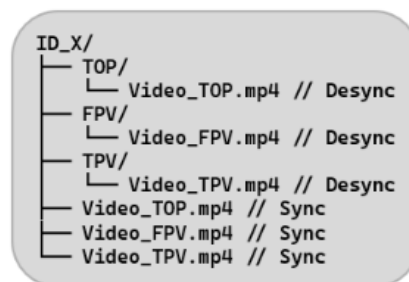


Figura 3.1: Struttura del dataset

All'interno del dataset è importante andare a distinguere la gestione delle telecamere in base al loro profilo di movimento per ottimizzare la stima della posa. Le viste TOP e TPV vengono configurate come telecamere statiche. Per identificare queste due tipologie di visuali statiche, è necessario includere la stringa "cam" nella nomenclatura delle cartelle durante la fase di preprocessing in cui avviene la frammentazione del video da formato MP4 in immagini in formato JPEG. Al contrario, la vista FPV è classificata come telecamera dinamica poiché, essendo indossata da un soggetto in attività, è soggetta a spostamenti continui e rotazioni repentine. In questo caso non

deve esserle inclusa la stringa "cam" nella nomenclatura della sua cartella. Prima di iniziare l'attività di sincronizzazione dei video, come accennato in precedenza, è stata necessaria una fase di preprocessing. Il framework non elabora direttamente i file video, ma richiede una trasformazione preventiva in sequenze di immagini. Questa trasformazione è stata realizzata attraverso script di preparazione dedicati, in cui i video originali del dataset vengono decodificati e campionizzati in  $N$  frame dove  $N$  è dato dalla seguente formula

$$N = \text{FPS} \times (\text{intervallo elaborato [s]})$$

I singoli frame estratti sono riorganizzati in maniera gerarchica come rappresentato nell'immagine seguente:

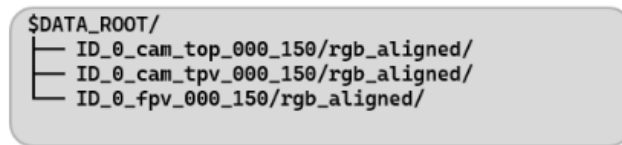


Figura 3.2: Struttura del dataset dopo il preprocessing

## 3.2 Modelli

Nella seguente sezione vengono riportati tutti i modelli citati all'interno del paper "VisualSync: Multi-Camera Synchronization via Cross-View Object Motion" [1].

### 3.2.1 Grounded-SAM 2

Grounded SAM 2 è un framework che combina Grounding DINO, un modello di object detection open-vocabulary in grado di localizzare oggetti in un'immagine a partire da una descrizione testuale, con SAM 2, Segment Anything Model 2, un modello di segmentazione esteso al dominio video grazie a un meccanismo di memoria temporale che ne consente il tracking coerente tra frame. La combinazione dei due moduli permette di ottenere maschere di segmentazione precise per oggetti specificati in linguaggio naturale, senza necessità di fine-tuning su classi predefinite, rendendo il modello particolarmente adatto a scenari multi-camera dove gli oggetti di interesse possono variare da sequenza a sequenza.

#### Grounding DINO

Il modello Grounding DINO è un sistema solido che è stato sviluppato per rilevare oggetti arbitrari specificati da input in linguaggio umano, un task comunemente denominato *open-set object detection*. Questo sistema è stato progettato basandosi su due principi: fusione stretta delle modalità basata su DINO, ovvero una fusione stretta tra modalità visiva e testuale, costruita sulla base dell'architettura DINO, e

pre-training grounded su larga scala per la generalizzazione di concetti mai visti esplicitamente come classe. Grounding DINO, dato in input una coppia immagine-testo, restituisce molteplici coppie bounding box-etichetta, dove l'etichetta corrisponde al sintagma nominale estratto dal testo in input. Grounding DINO è un'architettura dual-encoder-single-decoder: contiene un backbone per l'immagine per l'estrazione delle caratteristiche dell'immagine, un backbone per l'estrazione delle caratteristiche del testo, un potenziatore di caratteristiche per la fusione delle caratteristiche di immagine e testo, un modulo di selezione delle query guidato dal linguaggio per l'inizializzazione delle query e un decodificatore cross-modalità per il perfezionamento del riquadro. Per ogni coppia immagine-testo, vengono estratte caratteristiche standard dell'immagine e caratteristiche standard del testo utilizzando rispettivamente una backbone per le immagini e una per il testo. Le due caratteristiche standard vengono quindi inserite in un modulo di potenziamento delle caratteristiche per la fusione delle caratteristiche tra le diverse modalità. Dopo aver ottenuto le caratteristiche tra testo e immagine tra le diverse modalità, utilizziamo un modulo di selezione delle query guidato dal linguaggio per selezionare le query tra le diverse modalità dalle caratteristiche dell'immagine. Analogamente alle query sugli oggetti, nella maggior parte dei modelli di tipo DETR, Detection Transformers, queste query tra le diverse modalità vengono inserite in un decodificatore per le diverse modalità per analizzare le caratteristiche desiderate dalle due modalità e aggiornarsi automaticamente. Le query di output dell'ultimo livello del decodificatore vengono utilizzate per predire i riquadri degli oggetti ed estrarre le frasi corrispondenti. [2]

## SAM 2

Il Segment Anything Model 2 è un modello unificato per la segmentazione di video e immagini dove l'immagine è un video a singolo frame. Il modello produce maschere di segmentazione dell'oggetto di interesse, sia su singole immagini sia attraverso i frame video. SAM 2 è dotato di una memoria che immagazzina informazioni sull'oggetto e sulle interazioni precedenti, il che gli consente di generare predizioni di maschere spazio-temporali lungo tutto il video, e anche di correggerle efficacemente sulla base del contesto di memoria immagazzinato relativo all'oggetto, proveniente dai frame osservati in precedenza. SAM 2 può essere visto come una generalizzazione di SAM, infatti il modello ha un comportamento simile: un decoder di maschere promptable e leggero prende in input un embedding dell'immagine e dei prompt e restituisce una maschera di segmentazione per il frame. L'embedding del frame utilizzato dal decoder di SAM 2 non proviene direttamente da un encoder di immagini, ma è invece condizionato sulle memorie delle predizioni passate e dei frame con prompt. È possibile che i frame con prompt provengano anche dal *futuro* rispetto al frame corrente. Le memorie dei frame vengono create dal memory encoder sulla base della predizione corrente e collocate in una memory bank per l'uso nei frame successivi. L'operazione di memory attention prende l'embedding per-frame,

proveniente dall'immagine encoder, e lo condiziona sulla memory bank, prima che il mask decoder lo elabori per formare una predizione. Il modello è composto da diverse componenti:

- Image encoder: per l'elaborazione in tempo reale di video di lunghezza arbitraria, adottiamo un approccio streaming, consumando i frame video man mano che diventano disponibili. L'immagine encoder viene eseguito una sola volta per l'intera interazione, e il suo ruolo è fornire token non condizionati che rappresentano ciascun frame.
- Memory attention: il ruolo della memory attention è condizionare le feature del frame corrente sulle feature e predizioni dei frame passati, oltre che su eventuali nuovi prompt. Vengono disposti  $L$  blocchi transformer, il primo dei quali prende come input l'encoding dell'immagine del frame corrente. Ogni blocco esegue un self-attention, seguito da una cross-attention verso le memorie dei frame e verso i puntatori d'oggetto immagazzinati in una memory bank, ed infine un Multilayer Perceptron, MLP.
- Prompt encoder e mask decoder: il prompt encoder può essere attivato tramite click, box o maschere, per definire l'estensione dell'oggetto in un dato frame. I prompt sparsi sono rappresentati da encoding posizionali sommati a embedding appresi per ciascun tipo di prompt, mentre le maschere vengono incorporate tramite convoluzioni e sommate all'embedding del frame. Il design del decoder vede sovrapposti blocchi transformer *two-way* che aggiornano gli embedding dei prompt e del frame. Come in SAM, per prompt ambigui, prediciamo più maschere. Questo design è importante per garantire che il modello produca maschere valide. Nel video, dove l'ambiguità può estendersi attraverso più frame, il modello predice più maschere su ciascun frame. Se nessun prompt successivo risolve l'ambiguità, il modello propaga solo la maschera con l'IoU predetto più alto per il frame corrente. A differenza di SAM, dove esiste sempre un oggetto valido da segmentare dato un prompt positivo, è possibile che non esista alcun oggetto valido su alcuni frame e per questa nuova modalità di output, aggiungiamo una head che predice se l'oggetto di interesse è presente nel frame corrente o meno. Un'altra novità sono le skip connection dall'immagine encoder gerarchico per incorporare embedding ad alta risoluzione per il decoding delle maschere.
- Memory encoder: il memory encoder genera una memoria sottocampionando la maschera di output tramite un modulo convoluzionale e sommandola elemento per elemento con l'embedding non condizionato del frame proveniente dall'immagine encoder, seguito da layer convoluzionali leggeri per fondere l'informazione.
- Memory bank: la memory bank conserva informazioni sulle predizioni passate relative all'oggetto target nel video, mantenendo una coda FIFO di memorie



fino a  $N$  frame recenti, e immagazzina informazioni provenienti dai prompt in una coda FIFO fino a  $M$  frame con prompt. Oltre alla memoria spaziale, vengono memorizzati una lista di puntatori d'oggetto come vettori leggeri per l'informazione semantica ad alto livello dell'oggetto da segmentare, basati sui token di output del mask decoder di ciascun frame. La nostra memory attention esegue cross-attention sia verso le feature di memoria spaziale sia verso questi puntatori d'oggetto. Vengono inserite informazioni di posizione temporale nelle memorie degli  $N$  frame recenti, permettendo al modello di rappresentare il movimento a breve termine dell'oggetto, ma non in quelle dei frame con prompt, perchè il segnale di addestramento proveniente dai frame con prompt è più scarso ed è più difficile generalizzare al contesto di inferenza, dove i frame con prompt possono provenire da un intervallo temporale molto diverso rispetto a quello del training. [3]

### 3.2.2 DEVA

Il modello DEVA, per la segmentazione video disaccoppiata, è stato introdotto per ridurre la dipendenza dalla quantità di dati di addestramento specifici per il task, sfruttando dati esterni al dominio di destinazione. Il modello combina la segmentazione a livello di immagine specifica per il task e la propagazione temporale indipendente dal task stesso e, grazie a questa progettazione, si ha bisogno solo di un modello a livello di immagine per il task di destinazione e di un modello di propagazione temporale universale che viene addestrato una sola volta e si generalizza a diversi task. DEVA è guidato da un modello di segmentazione dell'immagine e da un modello di propagazione temporale universale: il modello di immagine, è stato addestrato specificamente per il task di target, fornendo ipotesi di segmentazione a livello di immagine specifiche per il task, mentre il modello di propagazione temporale, è stato addestrato su dataset di propagazione di maschere agnostiche rispetto alla classe, associando e propagando queste ipotesi per segmentare l'intero video. Questa progettazione separa l'apprendimento della segmentazione specifica per il task e l'apprendimento della segmentazione generale degli oggetti video, portando ad ottenere un framework robusto anche quando i dati nel dominio target scarseggiano e sono insufficienti per l'apprendimento end-to-end. Il sistema mira a propagare le segmentazioni scoperte dal modello di segmentazione dell'immagine all'intero video tramite la propagazione temporale. A partire dal primo frame, si utilizza il modello di segmentazione dell'immagine per l'inizializzazione. Per ridurre il rumore introdotto dalla segmentazione del singolo frame, DEVA analizza una breve clip di pochi fotogrammi futuri e genera per ciascuno una proposta di segmentazione. Le proposte vengono poi confrontate tra loro in base al grado di sovrapposizione reciproca: viene mantenuta come output solo la segmentazione più supportata dalle altre proposte della clip, mentre quelle isolate, prive di riscontro nei frame vicini, vengono scartate come rumore. Questo meccanismo, definito *consenso in-clip*, permette di ottenere

una segmentazione più stabile e coerente rispetto all'elaborazione indipendente dei singoli frame. Successivamente, viene utilizzato il modello di propagazione temporale per propagare la segmentazione ai frame successivi. Come modello di propagazione temporale, è stato utilizzato, con delle modifiche, *XMem*. Questo modello non è stato progettato per segmentare nuovi oggetti che appaiono nella scena. Pertanto, vengono integrati periodicamente i nuovi risultati di segmentazione dell'immagine utilizzando lo stesso consenso in-clip di prima per poi unire il consenso con il risultato propagato. Il consenso in-clip opera sulle segmentazioni dell'immagine di una piccola clip futura di  $n$  fotogrammi e produce un consenso senza rumore per il fotogramma corrente. In sintesi, prima si ottiene un insieme di proposte di oggetti sul fotogramma target tramite un allineamento spaziale, poi vengono unite le proposte di oggetti in una rappresentazione combinata ed infine, si ottimizza rispetto ad una variabile indicatrice per scegliere un sottoinsieme di proposte come output in un programma a variabili intere. Invece, per unire la segmentazione propagata con il consenso in un'unica segmentazione, vengono associati i segmenti di queste due segmentazioni ed è possibile osservare che, a differenza del consenso in-clip, queste segmentazioni contengono informazioni fondamentalmente diverse. Per questo motivo, non viene eliminato alcun segmento, ma vengono messe insieme tutte le coppie di segmenti associati, lasciando passare i segmenti non associati all'output. [4]

### 3.2.3 VGGT

Visual Geometry Grounded Transformer, VGGT, è una rete neurale feed-forward che esegue la ricostruzione 3D a partire da una, poche o persino centinaia di viste di una scena. VGGT predice un set completo di attributi 3D, parametri della telecamera, mappe di profondità, mappe dei punti e le tracce dei punti 3D. Il tutto avviene in un singolo passaggio in avanti, in pochi secondi. VGGT è un modello che può migliorare attività come il tracciamento dei punti nei video dinamici e la sintesi di nuove prospettive. Il modello, quindi, prende in input una sequenza di immagini RGB, che osservano la medesima scena tridimensionale da viste diverse. L'obiettivo del modello è inferire, per ciascuna immagine della sequenza, un insieme completo di annotazioni geometriche 3D, tramite un'unica funzione transformer applicata congiuntamente all'intera sequenza. Per ogni immagine il transformer produce quattro elementi:

- Parametri di camera: codificano sia i parametri intrinseci che estrinseci della camera, parametrizzati come concatenazione di: quaternione di rotazione, vettore di traslazione e campo visivo. Si assume che il punto principale della camera coincida con il centro dell'immagine.
- Mapa di profondità: associa a ciascun pixel dell'immagine il valore di profondità stimato rispetto alla camera  $i$ -esima.

- Point map: associa a ciascun pixel le coordinate del corrispondente punto 3D della scena. Questa caratteristica è fondamentale perché, queste point map, sono invarianti rispetto al punto di vista, cioè, tutti i punti 3D, indipendentemente da quale immagine li ha prodotti, vengono espressi in un unico sistema di riferimento globale, coincidente con quello della prima camera. Questo consente di mettere in corrispondenza diretta le ricostruzioni 3D ottenute da viste diverse.
- Keypoint tracking: dato un punto fisso dell'immagine di query nell'immagine di query, la rete produce una traccia formata dai corrispondenti punti 2D in tutte le immagini. Si noti che il transformer non produce le tracce direttamente, ma piuttosto le caratteristiche, che vengono utilizzate per il tracciamento.

Il modello è composto da un'architettura semplice con bias induttivi 3D minimi, consentendo al modello di apprendere da ampie quantità di dati annotati in 3D. In particolare, implementiamo il modello come un grande transformer, dove ogni immagine di input viene inizialmente divisa in patch tramite DINO. L'insieme combinato di token di immagine da tutti i frame, viene successivamente elaborato attraverso la struttura di rete principale, alternando livelli di autoattenzione frame-wise e globali. Nel design standard del transformer è stato introdotto l'Alternating-Attention, facendo sì che il transformer si concentri alternativamente all'interno di ciascun frame e globalmente. Il numero di token dipende dalla risoluzione dell'immagine, in particolar modo, l'attenzione a livello di singolo frame, si concentra sui token all'interno di ciascun frame separatamente, mentre l'attenzione globale si concentra sui token in tutti i frame congiuntamente. [5]

### 3.2.4 CoTracker3

CoTracker3 è un modello di tracciamento di punti che si basa sulle idee dei tracker più recenti ma è significativamente più semplice, più efficiente in termini di dati e più flessibile. Questo modello non introduce necessariamente nuovi componenti, ma dimostra che combinando elementi già noti in modo più semplice si ottengono prestazioni superiori con molti meno dati e training più semplici. CoTracker3 prende elementi da modelli precedenti, come aggiornamenti iterativi e le feature convoluzionali da PIPs, la cross-track attention per il tracciamento congiunto, le virtual track per l'efficienza, e l'unrolled training per l'operatività a finestra da CoTracker, oltre alla correlazione 4D da LocoTrack. Allo stesso tempo, semplifica alcuni di questi componenti e ne rimuove altri, come la fase di global matching di BootsTAPIR e LocoTrack. Ci sono due versioni del modello di CoTracker3: offline e online. La versione online opera tramite una sliding window, elaborando il video in input in modo sequenziale e tracciando i punti solo in avanti, mentre, la versione offline elabora l'intero video come un'unica sliding window, consentendo il tracciamento dei punti sia in avanti sia all'indietro. La versione offline traccia meglio

i punti occlusi e migliora anche il tracciamento a lungo termine dei punti visibili, tuttavia, il numero massimo di frame tracciabili è limitato dalla memoria, mentre la versione online può tracciare indefinitamente in tempo reale. Il funzionamento di CoTracker3 si articola in tre fasi principali:

- Feature maps: per ciascun frame video viene calcolata una feature map densa tramite una rete neurale convoluzionale. Il video viene sottocampionato di un fattore  $k=4$  per efficienza, e le feature vengono estratte a 4 scale differenti.
- Feature di correlazione 4D: per localizzare un punto di query in tutti i frame del video, il modello calcola la correlazione tra i vettori di feature attorno al punto di query nel frame iniziale e i vettori di feature attorno alle stime correnti della traccia negli altri frame. Queste correlazioni vengono poi proiettate tramite un MLP per ridurre la dimensionalità.
- Aggiornamento iterativo: le tracce, la confidenza e la visibilità dei punti vengono inizializzate e poi raffinate iterativamente da un transformer. Ad ogni iterazione, vengono incorporate le tracce utilizzando la codifica di Fourier degli spostamenti per fotogramma, dopodiché, vengono concatenati gli incorporamenti delle tracce, la confidenza, la visibilità e le correlazioni 4D per ogni punto di query. Questo forma una griglia di token di input per il transformer che copre il tempo e il numero di punti di query. Il transformer, una volta presa questa griglia come input, aggiunge gli embedding temporali di Fourier standard e applica l'attenzione temporale fattorizzata e l'attenzione di gruppo, inoltre per migliorare l'efficienza, utilizza dei token proxy. Il modello si occuperà di andare ad aggiornare, le tracce, la confidenza e la visibilità per un certo numero di volte. [6]

### 3.2.5 MAST3R

*Matching And Stereo 3D Reconstruction*, è un modello per l'immagine matching che affronta il problema della corrispondenza tra pixel come un task intrinsecamente 3D, anziché come un problema 2D nello spazio immagine, come avviene tradizionalmente. Il modello nasce come estensione di DUST3R, un framework basato su transformer per la ricostruzione 3D, che si è dimostrato robusto a forti cambi di punto di vista, ma impreciso quando i suoi output vengono utilizzati direttamente per il matching. MAST3R supera questo limite aggiungendo a DUST3R una seconda testa di predizione che produce feature locali dense, addestrata con una loss di matching dedicata, preservando così la robustezza del framework originale mentre ne migliora sensibilmente l'accuratezza. Il modello affronta anche il problema della complessità quadratica del matching denso, introducendo uno schema di matching reciproco veloce che accelera il processo di diversi ordini di grandezza, fornendo al contempo garanzie teoriche di convergenza e migliorando la qualità dei risultati finali. Date due immagini in input, il modello codifica ciascuna immagine con dei pesi condivisi tramite un

encoder ViT, ottenendo due rappresentazioni separate. Queste rappresentazioni vengono poi elaborate congiuntamente da due decoder intrecciati, che si scambiano informazioni tramite cross-attention per comprendere la relazione spaziale tra i punti di vista e la geometria 3D globale della scena. Per ottenere corrispondenze pixel-accurate in tempi ragionevoli, MAST3R introduce due meccanismi aggiuntivi rispetto al semplice matching denso:

- Fast reciprocal matching: il matching reciproco denso, ovvero la ricerca dei vicini più prossimi reciproci tra le mappe di feature delle due immagini, ha una complessità computazionale quadratica rispetto al numero di pixel, risultando proibitivo per applicazioni pratiche. Per questo motivo, MAST3R introduce un algoritmo iterativo che parte da un piccolo insieme di pixel campionati su una griglia regolare nella prima immagine, li proietta nei rispettivi vicini più prossimi nella seconda immagine, e poi proietta nuovamente questi punti nella prima immagine, verificando se si formano dei cicli. I punti che hanno già raggiunto una corrispondenza stabile vengono rimossi dalle iterazioni successive, mentre il processo continua per i restanti, fino a convergenza. Questo schema risulta quasi due ordini di grandezza più veloce rispetto al matching denso completo, pur fornendo garanzie teoriche di convergenza e, sorprendentemente, migliorando anche l'accuratezza finale del matching.
- Coarse-to-fine matching: poiché la complessità quadratica dell'attenzione rispetto all'area dell'immagine limita MAST3R a lavorare con immagini di al più 512 pixel sul lato maggiore, le immagini ad alta risoluzione devono essere gestite con uno schema a grana progressiva. Il matching viene inizialmente eseguito su versioni ridotte delle due immagini per ottenere un insieme di corrispondenze 'grossolane'. Successivamente, vengono generate griglie di ritagli sovrapposti sulle immagini a piena risoluzione, selezionando un sottoinsieme di coppie di finestre che copre la maggior parte delle corrispondenze grossolane individuate. Il matching viene quindi rieseguito indipendentemente su ciascuna coppia di finestre, e le corrispondenze ottenute vengono infine rimappate alle coordinate originali e concatenate, fornendo corrispondenze dense a piena risoluzione. [7]

### 3.3 Le tre fasi di visualsync

L'obiettivo di VisualSync è quello di sfruttare i recenti progressi nella computer vision, in particolare il tracciamento denso, le corrispondenze tra viste e la ricostruzione 3D robusta da movimento, per costruire un sistema robusto e versatile in grado di sincronizzare in modo affidabile video complessi. Nello specifico, è stata formulata una funzione di energia congiunta che misura le violazioni dei vincoli epipolari tra le corrispondenze tra i video ed è stata adottata una procedura di ottimizzazione a tre fasi. Nella Fase 0, viene utilizzato il modello VGGT per stimare le matrici fondamentali tra ciascuna coppia di video, in seguito viene applicato il modello

MAST3R per stabilire le corrispondenze tra i video ed infine CoTracker3 che viene utilizzato per stabilire tracce dense all'interno di ciascun video. Questo dà accesso alle quantità necessarie per valutare l'energia congiunta, infatti, nella Fase 1, questa energia viene scomposta in termini energetici a coppie e viene stimato il miglior allineamento temporale tra ciascuna coppia di video tramite una ricerca esaustiva. Infine, nella Fase 2, vengono sincronizzati gli offset temporali tra tutte le coppie di video in modo da assegnare un offset temporale globale coerente a ciascun video.

### **3.3.1 Fase 0: Estrazione degli indizi visivi**

Il primo passaggio è la segmentazione del movimento, in cui vengono identificati gli oggetti dinamici nella scena. VisualSync consente di fornire direttamente i frame video in input a GPT-4o in modo tale da identificare le classi di oggetti effettivamente in movimento nella scena. Le classi individuate da GPT-4o vengono poi utilizzate per guidare Grounding DINO, che ha come obiettivo quello di generare i bounding box corrispondenti a tali classi. Questi bounding box vengono infine forniti come prompt a SAM 2, che va a generare le maschere di segmentazione precise degli oggetti dinamici. Infine, per garantire una segmentazione temporale coerente lungo l'intero video, viene applicato DEVA, che va a tracciare le maschere. Il secondo passaggio riguarda la corrispondenza spazio-temporale. Per catturare le traiettorie di movimento degli oggetti dinamici, viene applicato CoTracker3 a ciascun video all'interno della rispettiva regione dinamica individuata, producendo così un insieme di tracklet 2D che rappresentano la traiettoria osservata nello spazio immagine di ciascun punto dinamico nel tempo. Per stabilire corrispondenze tra le tracklet di video diversi, viene poi eseguito un matching cross-view tramite MAST3R, dove, per ogni tracklet di un video, viene campionato un sottoinsieme di keyframe e viene calcolata la similarità a coppie con i keyframe campionati degli altri video, costruendo così coppie candidate di tracklet che condividono lo stesso punto 3D dinamico osservato da viste diverse. Il terzo passaggio è il filtraggio delle corrispondenze, necessario per andare a sopprimere il rumore introdotto dai passaggi precedenti. Sfruttando il matching di istanza fornito da DEVA, viene costruita una matrice di corrispondenze a coppie, andando a contare quante corrispondenze vengono trovate tra ciascuna coppia di istanze su un sottoinsieme di frame campionati. Viene poi applicato un matching bipartito per ottenere assegnazioni ottimali uno-a-uno tra le istanze nei due video, scartando le coppie con meno di 100 corrispondenze. Solo le corrispondenze di MAST3R, i cui estremi appartengono alla stessa istanza così accoppiata, vengono mantenute, e tutte le corrispondenze rimanenti, vengono aggregate in un unico insieme di match spazio-temporali, che andrà a costituire l'input per il processo di sincronizzazione basato sull'energia. L'ultimo passaggio è la stima dei parametri di camera, affidata al modello VGGT, il quale si occupa di andare ad estrarre pose per ciascuna camera nella pipeline di preprocessing. Per le camere statiche viene fornito in input solo il primo frame, mentre, per le camere dinamiche, vengono utilizzati tutti

i frame disponibili. Gli output di VGGT comprendono, per ciascun video, parametri estrinseci per fotogramma e parametri intrinseci per ogni video. Questi parametri vengono quindi utilizzati per calcolare le pose relative e le matrici fondamentali.

### 3.3.2 Fase 1: Stima degli offset a coppie

Nella Fase 1, VisualSync, rilassa il vincolo dell'ottimizzazione globale e cerca, per ciascuna coppia di camere  $(i, j)$ , l'offset temporale ottimale  $\Delta^{ij}$  all'interno di un insieme discreto e finito di candidati  $\mathcal{S}$ , tramite ricerca esaustiva:

$$\Delta^{ij*} = \arg \min_{\Delta \in \mathcal{S}} E_{ij}(\Delta) \quad (3.1)$$

La dimensione del passo dell'insieme  $\mathcal{S}$ , è determinata dal frame rate del video, mentre l'intervallo di ricerca è un iperparametro del metodo. In questo modo, ciascun termine di energia a coppie, può essere minimizzato in modo indipendente dagli altri. Il cuore della fase 1 di VisualSync, è la definizione del termine di energia  $E_{ij}(\Delta)$ , che va a quantificare quanto le tracklet associate violano il vincolo epipolare per un dato candidato di offset  $\Delta$ . Tra le diverse misure di errore epipolare disponibili, VisualSync adotta l'errore di Sampson, che va ad approssimare la distanza euclidea al quadrato tra un punto e la sua retta epipolare corrispondente, ed è ottenuto linearizzando il vincolo epipolare attorno all'osservazione corrente. Più nel dettaglio, sia  $\mathbf{x}_t = [\mathbf{x}^i(t + \Delta); \mathbf{x}^j(t)]$  una corrispondenza spazio-temporale rumorosa, e sia  $\mathbf{z}_t$  la corrispondente osservazione "pulita" sottostante, che soddisfa esattamente il vincolo epipolare:

$$C(\mathbf{z}_t) = \mathbf{z}^i(t + \Delta)^\top \mathbf{F}_{t+\Delta, t}^{ij} \mathbf{z}^j(t) = 0 \quad (3.2)$$

dove  $\mathbf{F}_{t+\Delta, t}^{ij}$  è la matrice fondamentale tra la camera  $i$  al tempo  $t + \Delta$  e la camera  $j$  al tempo  $t$ , che codifica il vincolo epipolare tra le due viste: dati due punti corrispondenti nelle due immagini, il prodotto  $\mathbf{x}^{i\top} \mathbf{F}^{ij} \mathbf{x}^j$  è uguale a zero se e solo se i punti soddisfano la geometria epipolare, ovvero se e solo se i due video sono correttamente sincronizzati. L'errore di Sampson nasce dal voler quantificare la correzione minima necessaria per proiettare l'osservazione rumorosa  $\mathbf{x}_t$  sulla varietà epipolare definita da questo vincolo:

$$E^2 = \min_{\mathbf{z}_t} \|\mathbf{z}_t - \mathbf{x}_t\|^2 \quad \text{s.t. } C(\mathbf{z}_t) = 0 \quad (3.3)$$

Poiché questo problema di ottimizzazione vincolata è non lineare, e quindi difficile da risolvere direttamente, lo si approssima linearizzando il vincolo  $C$  attorno a  $\mathbf{x}_t$  tramite uno sviluppo di Taylor al primo ordine. Risolvendo per la correzione ottima si ottiene l'approssimazione di Sampson dell'energia, che fornisce un limite inferiore al primo ordine sull'energia originale e che, a differenza della formulazione vincolata di partenza, ammette un'espressione in forma chiusa, rendendola computazionalmente

efficiente.

$$\mathcal{E}_{\text{Sampson}}^2 = \frac{\left(\mathbf{x}^i(t + \Delta)^\top \mathbf{F}_{t+\Delta,t}^{ij} \mathbf{x}^j(t)\right)^2}{\|\mathbf{F}_{t+\Delta,t}^{ij} \mathbf{x}^j(t)\|_{1,2}^2 + \|\mathbf{F}_{t+\Delta,t}^{ij\top} \mathbf{x}^i(t + \Delta)\|_{1,2}^2} \quad (3.4)$$

Il termine di energia complessivo, sommato su tutte le coppie di tracklet  $(\mathbf{x}^i, \mathbf{x}^j)$  co-visibili tra le camere  $i$  e  $j$  e su tutti i frame  $t$ , è il seguente:

$$E_{ij}(\Delta) = \sum_{(\mathbf{x}^i, \mathbf{x}^j)} \sum_t \frac{\left(\mathbf{x}^i(t + \Delta)^\top \mathbf{F}_{t+\Delta,t}^{ij} \mathbf{x}^j(t)\right)^2}{\|\mathbf{F}_{t+\Delta,t}^{ij} \mathbf{x}^j(t)\|_{1,2}^2 + \|\mathbf{F}_{t+\Delta,t}^{ij\top} \mathbf{x}^i(t + \Delta)\|_{1,2}^2} \quad (3.5)$$

Il numeratore rappresenta il residuo epipolare algebrico al quadrato, cioè quanto la corrispondenza osservata viola il vincolo epipolare, mentre il denominatore va a sommare le lunghezze al quadrato delle normali delle due rette epipolari dove,  $\|\mathbf{F}_{t+\Delta,t}^{ij} \mathbf{x}^j(t)\|_{1,2}^2$  è la norma della retta epipolare di  $\mathbf{x}^j(t)$  proiettata nella vista  $i$ , e  $\|\mathbf{F}_{t+\Delta,t}^{ij\top} \mathbf{x}^i(t + \Delta)\|_{1,2}^2$  è la norma della retta epipolare di  $\mathbf{x}^i(t + \Delta)$  proiettata nella vista  $j$ . Questa normalizzazione converte il residuo grezzo in un'approssimazione della distanza punto-retta al quadrato, molto vicina al vero errore di reproiezione, pur mantenendo un calcolo rapido e in forma chiusa.

Non tutte le coppie di camere producono però stime affidabili dell'offset temporale: alcune coppie hanno una sovrapposizione temporale minima, altre una sovrapposizione di viewpoint insufficiente, oppure gli indizi visivi estratti nella Fase 0 possono essere rumorosi. Per questo motivo, VisualSync scarta automaticamente le coppie il cui rapporto tra l'energia ottima e il secondo minimo locale migliore scende sotto 0.1, oppure che presentano più di due minimi locali, ottenendo così un sottoinsieme di coppie affidabili da utilizzare nella fase successiva di sincronizzazione globale.

### 3.3.3 Fase 2: Stima globale degli offset

L'obiettivo della Fase 2 è ricostruire gli offset globali  $\{s^i\}$  per tutti i video a partire dalle stime a coppie  $\Delta^{ij}$  ottenute nella Fase 1. Poiché queste stime sono discrete e imperfette, in generale non esiste una soluzione  $\{s^i\}$  che soddisfi esattamente  $s^j - s^i = \Delta^{ij}$  per ogni coppia affidabile  $(i, j) \in \mathcal{E}$ . Per questo motivo, VisualSync formula il problema come un problema di minimi quadrati robusti:

$$\{s^i\}^* = \arg \min_{\{s^i\}} \sum_{(i,j) \in \mathcal{E}} \rho_\delta \left(s^j - s^i - \Delta^{ij}\right) \quad (3.6)$$

dove  $\rho_\delta$  è la **loss di Huber**. Il problema viene risolto tramite una procedura di minimi quadrati iterativamente ripesati, ottenendo come risultato finale gli offset globali di sincronizzazione  $\{s^i\}^*.[1]$



## Capitolo 4

### Miglioramenti e ottimizzazioni architettureali

L'obiettivo del lavoro è quello di testare il modello VisualSync sul dataset fornito dal docente per verificare la corretta sincronizzazione delle camere poste in punti diversi che riprendono la stessa scena. Dopo un numero elevato di test seguendo la pipeline ufficiale di VisualSync, è emerso in modo evidente, come l'andare ad eseguire molteplici comandi in sequenza, per poi ripeterli in maniera identica per ogni nuovo ID del dataset, fosse dispendioso in termini di tempo. Abbiamo quindi definito un insieme di script pensati per ottimizzare l'esecuzione velocizzando e rendendo ripetibile sia l'esecuzione del modello, sia la successiva fase di valutazione dei risultati, distinguendo due livelli di automazione: uno relativo all'esecuzione della pipeline vera e propria, l'altro relativo alla validazione quantitativa dei suoi output.

Il primo passo è stato lo sviluppo dello script Python `exec_visualsync.py`, che si occupa di riportare tutti i passi definiti dal framework, in un'unica esecuzione, passando dal cropping temporale del dataset grezzo fino al calcolo degli offset globali e alla generazione del video sincronizzato multi-vista. Prima di lanciare lo script, è stato necessario verificare che sia attivo il venv, in quanto tutti gli script fanno riferimento a librerie che risiedono in questo ambiente. Una volta verificato, viene lanciato lo script dalla cartella principale del progetto attraverso il comando `python src/custom_scripts/exec_visualsync.py`. Il primo risultato del lancio dello script, è quello di proporre a schermo una sequenza di prompt interattivi con cui l'utente inserisce: l'ID o gli ID del dataset su cui lavorare, in modo da eseguire automaticamente l'intera pipeline su più ID (o su un ID) in sequenza senza dover reinserire manualmente le impostazioni per ciascuno, i parametri temporali su cui focalizzare l'analisi video indicando l'inizio dell'intervallo del video, `start_sec`, e la fine `end_sec` ed infine la frequenza di `fps`. Dopo questa prima interazione con l'utente, lo script chiede anche il passo da cui eventualmente riprendere l'esecuzione, in modo tale da gestire in modo efficiente le interruzioni dovute a errori intermedi, evitando così di dover ripetere dall'inizio fasi già completate correttamente. Al termine dell'esecuzione, lo script stampa a schermo anche i tempi impiegati e i dati relativi al consumo energetico e alle emissioni di anidride carbonica stimate, così da

poter correlare la durata di ciascun esperimento con il relativo impatto energetico oltre che con la qualità dei risultati ottenuti.

Un secondo gruppo di script è stato sviluppato per automatizzare il calcolo e la verifica delle metriche di sincronizzazione, a partire dall'esigenza di confrontare in modo sistematico l'output del modello con un ground truth affidabile, evitando di dover ricalcolare manualmente le durate e gli offset per ogni soggetto testato. Lo script Python `ground_truth_extractor.py` va ad elaborare le durate e i contenuti sia dei video originali, sia dei video sincronizzati presenti nel dataset per ricavare gli offset di riferimento, sia pairwise che quelli globali, da utilizzare come ground truth nella valutazione dell'errore del modello. È importante osservare che la vista TPV viene presa come vista di riferimento in questo modo tutti gli offset temporali delle restanti camere, ovvero la vista TOP e la vista FPV, sono stati calcolati in relazione alla linea temporale della TPV. La versione iniziale di questo script prevedeva di sfruttare le sole durate dei video originali e di quelli sincronizzati, facendone la differenza per calcolare gli offset di ground truth, ma il risultato non era corretto a causa delle approssimazioni temporali e della staticità di alcune riprese. Nella versione finale, l'algoritmo analizza i video originali calcolando la differenza tra due frame consecutivi per catturare la "dinamica" all'interno del video. Per ottimizzare la precisione e l'efficienza della ricerca, viene isolato un momento di forte discontinuità visiva, ovvero dove c'è un picco di variazione nei pixel che identifica un movimento netto e univoco. Una volta individuato e tracciato questo cambiamento chiave all'interno del video sincronizzato, il calcolo degli offset reali è risultato estremamente preciso e corretto. Lo script può essere lanciato seguendo il comando `python src/custom_scripts/ground_truth_extractor.py -root "$RAW_ROOT" -ids ID_0 ID_1`. Lo script `estimate_metrics.py` si occupa di confrontare i valori di ground truth generati da `ground_truth_extractor.py` con l'offset stimato dal modello per un dato ID, andando a calcolare l'errore medio e mediano in millisecondi, l'accuratezza a due soglie di tolleranza, A100ms e A500ms, espresse come percentuale di viste il cui errore di sincronizzazione ricade al di sotto della rispettiva soglia e l'Area Under Curve (AUC) come sintesi complessiva della qualità della stima. Per poter mandare in esecuzione lo script deve essere eseguito il comando `python src/custom_scripts/estimate_metrics.py`, che, una volta eseguito, chiede di inserire l'ID e il valore dell'*fps* su cui basare il test. Lo script `video_inspector.py` consente di ispezionare visivamente i video originali del dataset, mettendo a confronto i frame delle tre viste TOP, TPV, FPV in corrispondenza degli istanti temporali di interesse. Questo script è stato impiegato principalmente come strumento diagnostico per verificare visivamente la correttezza del ground truth calcolato da `ground_truth_extractor.py`. Anche questo script viene lanciato con il comando `python src/custom_scripts/video_inspector.py`. Per poter ulteriormente fornire uno strumento di ottimizzazione, abbiamo introdotto lo script `run_full_validation.py` per lanciare con un unico comando le 3

esecuzioni descritte in precedenza, in modo tale da calcolare il ground truth di un dato ID tramite `ground_truth_extractor.py`, passare poi i valori a `video_inspector.py` per un controllo di coerenza visiva, e infine fornire gli stessi dati a `estimate_metrics.py` per il calcolo dell'errore complessivo. Il comando di lancio è `python src/custom_scripts/run_full_validation.py`, anch'esso condizionato al preventivo settaggio delle variabili globali.

A monte di tutti gli script sopra descritti si colloca `set_global_variables.sh`, che imposta in un'unica operazione l'insieme delle variabili d'ambiente necessarie all'esecuzione: *RAW\_ROOT*, in cui va indicato il percorso del dataset PRIN, *GROUP*, in cui va inserito l'identificativo del soggetto, *START\_SEC* e *END\_SEC*, in cui va indicata la finestra temporale, *FPS*, in cui vengono inseriti i frame rate di campionamento, le cartelle derivate *DATA\_ROOT*, *TRACK\_ROOT* e *RESULT\_ROOT*, il prefisso delle maschere di segmentazione *MASK\_PREFIX*, oltre al *PYTHONPATH* esteso con i percorsi di MAST3R e DUST3R. Questo script viene richiamato con il seguente comando `source scripts/custom_scripts/set_globals_variables.sh ID_<gruppo> <start_sec> <end_sec> <fps>`. Tutti gli altri script dipendono dalla definizione di queste variabili globali senza le quali non verrebbero eseguiti correttamente.



# Capitolo 5

## Risultati finali

### 5.1 Iperparametrizzazione dell'offset range

L'`offset_range` è un parametro che definisce l'ampiezza, espressa in frame, della finestra di ricerca entro cui l'algoritmo valuta i possibili disallineamenti temporali tra una coppia di viste. Ovvero, dato un valore di `offset_range = k`, il modello considera come candidati tutti gli offset interi compresi nell'intervallo  $[-k, +k]$  attorno allo zero, calcolando per ciascuno una misura di coerenza geometrica tra le corrispondenze tracciate nelle due viste, basata sulla distanza di Sampson, e andando a selezionare, come offset stimato, quello che minimizza tale misura. Per poter ottenere buoni risultati è consigliato impostare un offset range non elevato, in quanto, lavorando con intervalli di ricerca più ampi, possono essere introdotti introdurre falsi minimi, ovvero candidati che risultano numericamente più coerenti del vero offset ma che non corrispondono alla reale corrispondenza temporale tra le due sequenze.

Durante i nostri test, abbiamo scelto di confrontare i risultati ottenuti basandoci sull'assegnazione di due valori all'offset range, `offset_range = 30` e `offset_range = 40`. Abbiamo selezionato questi due valori al fine di analizzare la variazione dei risultati ottenuti utilizzando due valori di offset range tra loro caratterizzati da una differenza contenuta. Dal confronto è emerso che, per tre dei soggetti testati, ID\_0, ID\_1 e ID\_5, l'ampliamento della finestra di ricerca da 30 a 40 non produce alcuna variazione né nell'errore delle viste TOP e FPV né nell'AUC. Questo indica che, per questi casi, il minimo individuato dall'algoritmo entro un raggio di 30 frame rimane il minimo globale anche quando la finestra di ricerca viene ulteriormente estesa, a conferma della stabilità della stima in presenza di una corrispondenza sufficientemente marcata tra le viste. Per i restanti soggetti, invece, passare da un `offset_range = 30` ad un `offset_range = 40`, ha effetti eterogenei e complessivamente peggiorativi. Osservando la media dei valori ottenuti, l'errore medio sulla vista TOP passa da 681,25 ms con un `offset_range = 30` a 997,08 ms con un `offset_range = 40`, mentre l'errore medio sulla vista FPV passa da 1.231,34 ms a 1.294,38 ms, evidenziando come una camera dinamica genera più errore rispetto ad una camera statica. Possiamo inoltre osservare come anche la media globale dell'errore con un `offset_range = 30` è pari 956,29 ms, mentre con

un `offset_range = 40` aumenta a 1145,73 ms. L'AUC media si attesta a 0,6080 con `offset_range = 30`, mentre con `offset_range = 40` assume un valore pari ad 0,5315. Ancora più significativo è il confronto sulla deviazione standard dell'AUC, che passa da 0,1702 con un `offset_range = 30` a 0,2303 `offset_range = 40` ad indicare che l'ampliamento della finestra di ricerca non solo abbassa la prestazione media, ma aumenta sensibilmente anche la dispersione dei risultati tra i soggetti testati.

Dal grafico è possibile notare come il caso di ID\_2 con `offset_range = 40` costituisce un esempio diretto del fenomeno dei falsi minimi, perchè, ampliando la finestra di ricerca da 30 a 40 frame, l'algoritmo ha probabilmente individuato, all'interno della coppia TOP-TPV, una corrispondenza che è geometricamente più coerente in un intorno più distante dal vero offset, ma che il criterio di minimizzazione della distanza di Sampson non è in grado di distinguere dal minimo corretto. Il risultato è una stima dell'offset numericamente valida ma semanticamente errata, che si riflette in un errore di sincronizzazione dell'ordine di oltre 2 secondi e in un'AUC praticamente nulla. Questo caso dimostra quanto affermato all'inizio della sezione, ovvero che la scelta di ampi intervalli può indurre a falsi minimi.

## 5.2 Valutazioni per i diversi FPS

Durante il lavoro abbiamo svolto dei test a 10 fps, a 20 fps e a 30 fps. Per ogni soggetto che abbiamo preso in esame abbiamo calcolato la media e la deviazione standard dell'AUC, oltre all'errore medio e all'errore medio totale sulle singole viste. Nelle 3 diverse configurazioni possiamo notare che il valore dell'AUC è pari a 0,5056 lavorando a 10 fps, è poi pari a 0,5009 lavorando a 20 fps, ed infine è pari a 0,5513 lavorando a 30 fps, che rappresenta il valore più alto tra le tre configurazioni. La deviazione standard dell'AUC, invece, subisce delle piccole variazioni nelle tre configurazioni ad indicare che l'aumento del frame rate, pur migliorando leggermente la prestazione media, non riduce la dispersione dei risultati tra i diversi soggetti. Per quanto riguarda l'errore medio sulla vista TOP si mantiene sostanzialmente stabile ed è comunque elevato nelle tre configurazioni a differenza dell'errore medio sulla vista FPV che invece mostra un andamento diverso, ovvero, si riduce passando da 2.435,54 ms a 1.566,57 ms lavorando rispettivamente a 10 e a 20 fps, per poi risalire parzialmente a 1.877,21 ms nella configurazione a 30 fps. Da questi valori abbiamo poi ricavato l'errore medio totale, che passa da un valore pari a 1.642,77 ms a 10 fps, per poi passare ad un valore pari a 1.283,29 ms a 20 fps, e nell'ultima configurazione assume un valore pari a 1.434,44 ms a 30 fps. Nel complesso possiamo affermare che i test condotti sulle tre configurazioni di frame rate, delineano un quadro in cui il guadagno prestazionale legato all'aumento dell'fps è reale ma limitato. Il passaggio a 30 fps produce il valore di AUC media più alto tra le tre configurazioni testate e

il secondo errore medio totale più basso, risultando conforme con quanto affermato nella letteratura di VisualSync[1].

### 5.3 Risultati generali

A seguito di numerosi test per valutare la configurazione migliore in cui VisualSync offre le sue migliori prestazioni, siamo giunti alla conclusione che selezionando un frame rate pari a 30 e un offset range pari a 30, si ottengono i risultati mostrati in figura. Per analizzare i risultati generali ottenuti, è utile confrontarli con i risultati riportati nel paper originale di VisualSync [1] sui quattro dataset di validazione impiegati dagli autori: Egohumans, CMU Panoptic, 3D-POP e UDBD. Questo confronto ci permette di stabilire se le difficoltà osservate siano imputabili a limiti intrinseci del framework o, piuttosto, a caratteristiche specifiche del dataset fornito. Sui quattro dataset originali, VisualSync riporta un errore medio pari a 122,2 ms per Egohumans, 112,6 ms CMU Panoptic, 114,7 ms per 3D-POP e 20,2 ms per UDBD, con un errore mediano ancora più contenuto, ovvero 46,6 ms per Egohumans, 41,5 ms per CMU Panoptic, 77,8 ms per 3D-POP e 5,9 ms per UDBD. Nel PRIN dataset, nella configurazione a 30 fps, l'errore medio sale a 963,98 ms e l'errore mediano a 533,35 ms. Questi valori sono di un ordine di grandezza superiori rispetto al peggiore dei quattro dataset di riferimento e fino a due ordini di grandezza superiori rispetto al caso migliore. Il medesimo divario emerge dalle metriche di accuratezza a soglia. L'accuratezza A100ms varia nel dataset Egohumans è pari a 33,91%, nel dataset CMU Panoptic è pari al 26,0%, nel dataset 3D-POP è pari al 33,3% e nel dataset UDBD è pari all'82,1%. Nel PRIN dataset, l'accuratezza A100ms, si attesta invece all'11,11%, un valore inferiore al risultato peggiore ottenuto dai quattro dataset. L'accuratezza A500, nel PRIN dataset, pari a 53,13%, risulta paragonabile a quella ottenuta da CMU Panoptic, 51,2%, e da Egohumans, 55,8%, pur restando nettamente inferiore ai risultati ottenuti da 3D-POP, 69,3%, e UDBD, 94,3%. Dai risultati che abbiamo ottenuto, è possibile notare come, il modello VisualSync, applicato al PRIN dataset, con una soglia di 100 ms non raggiunge un livello di precisione simile ai risultati ottenuti negli altri dataset di validazione, mentre con una tolleranza più ampia, ovvero pari a 500 ms, le prestazioni si avvicinano a quelle osservate negli altri quattro dataset di riferimento. I risultati ottenuti sono coerenti con le ipotesi fatte precedentemente, secondo cui le prestazioni ottenute sul PRIN dataset siano fortemente condizionate da caratteristiche specifiche di alcune sequenze, le quali si discostano dalle condizioni presenti nei dataset multi-camera utilizzati per la validazione originale del framework, e che contribuiscono in modo sproporzionato a innalzare l'errore medio complessivo pur non impedendo il raggiungimento di una sincronizzazione entro una tolleranza più ampia.

## 5.4 Risultati CodeCarbon

Al fine di approfondire ulteriormente la valutazione delle prestazioni del modello, abbiamo deciso di monitorare anche l'impatto ambientale delle esecuzioni, stimando il consumo energetico e le emissioni di CO<sub>2</sub> ad esso associate mediante l'impiego della libreria *CodeCarbon*. Il grafico delle emissioni e dell'energia per esecuzione mostra un andamento non uniforme, con valori di energia consumata compresi grosso modo tra 0,83 kWh e 1,49 kWh e valori di emissioni tra circa 0,28 kg e 0,49 kg di CO<sub>2</sub>. È interessante notare come, in corrispondenza delle esecuzioni ID\_11 e ID\_12, le due grandezze mostrino un andamento discordante. ID\_11 presenta il picco massimo di energia consumata, circa 1,49 kWh, a fronte di un'emissione di CO<sub>2</sub> solo moderatamente elevata, mentre ID\_12 presenta il valore minimo di energia consumata, circa 0,83 kWh, associato a una delle emissioni più alte registrate, circa 0,49 kg. Dal grafico è possibile osservare che l'energia media utilizzata per esecuzione è pari a 0,90 kWh, con un'emissione media di 0,30 kg di CO<sub>2</sub>. Inoltre il totale complessivo delle emissioni generate dall'insieme delle esecuzioni sperimentali condotte nel corso del progetto ammonta a 13,85 kg di CO<sub>2</sub>.

Per poter assegnare un valore ai risultati forniti da *CodeCarbon*, abbiamo scelto di confrontarli con alcune situazioni quotidiane. Una singola richiesta testuale a un modello di AI, comporta una quantità di emissioni che vanno da 0,002 kg di CO<sub>2</sub> a 0,005 kg di CO<sub>2</sub>, mentre una ricerca su Google ne comporta circa 0,0002 kg. La generazione di un'immagine tramite un modello di AI, pur potendo arrivare fino a 1,5/1,6 kg di CO<sub>2</sub>, si colloca su un ordine di grandezza superiore rispetto alla singola esecuzione del nostro modello. Un ulteriore confronto è quello con la mobilità su strada [8], in quanto, percorrere un chilometro con un'autovettura a benzina, diesel o GPL in Europa, comporta emissioni stimate tra 0,23 e 0,25 kg di CO<sub>2</sub>, un valore molto vicino a quello di una singola esecuzione della pipeline.



# Capitolo 6

## Conclusioni

L'obiettivo del nostro lavoro è stato quello di sviluppare una metodologia per la sincronizzazione temporale di viste acquisite da più telecamere, poste in punti di visualizzazione diversa. I risultati che abbiamo ottenuto dicono che impostando un range offset pari a 30, piuttosto che a 40, vengono offerte prestazioni migliori da parte del modello. Questo comportamento è riconducibile al fatto che un range eccessivamente ampio aumenta la probabilità di individuare falsi minimi durante la ricerca del minimo durante la Fase 1, compromettendo la corretta minimizzazione dell'energia epipolare e portando a stime dell'offset temporale non affidabili. La scelta di un offset range più contenuto consente quindi di vincolare lo spazio di ricerca in modo più efficace, riducendo l'ambiguità nella stima pairwise. Una delle sperimentazioni che abbiamo effettuato, è stata quella di conoscere il limite che la GPU utilizzata raggiunge nella gestione della memoria. Infatti a seguito di vari test siamo giunti alla conclusione che le prestazioni si mantengono stabili fino a sequenze di 250 frame. Una volta superato questo limite, viene generato un errore nominato Out Of Memory che ferma l'esecuzione. Abbiamo inoltre condotto un'analisi comparativa utilizzando 3 diverse frequenze di acquisizione: 10fps, 20fps e 30 fps. I risultati a cui siamo giunti è che operando a 30fps si ottengono risultati migliori rispetto alle altre due modalità in quanto, viene fornita una maggiore densità di osservazioni temporali, migliorando la qualità delle tracklet e la precisione della stima degli offset. I risultati ottenuti evidenziano come il modello non riesca ad attestarsi entro la soglia dei 100 ms di errore, mentre la percentuale di coppie sincronizzate entro i 500 ms risulta comparabile, se non superiore, ai risultati ottenuti sugli altri dataset citati nel paper originale di VisualSync [1]. Si osserva inoltre come il dataset PRIN presenti le medesime criticità già riscontrate sul dataset Egohumans, riconducibili alla presenza di camere dinamiche.



## Capitolo 7

### Possibili sviluppi futuri

I risultati a cui siamo arrivati ci suggeriscono alcuni temi che sarebbero utili da approfondire. Il primo riguarda l'ottimizzazione del modello mediante fine-tuning che potrebbe favorire un miglioramento dell'accuratezza nella stima degli offset temporali, incrementare la capacità di generalizzazione del modello e renderlo più robusto rispetto alle variazioni presenti nei dati, contribuendo così a ottenere prestazioni complessivamente superiori. Un secondo tema che sarebbe interessante andare ad approfondire è quello di individuare un range di offset ottimale. Sarebbe utile definire delle tecniche che vadano a calcolare un valore ottimale da assegnare al range offset in modo tale che non venga scelto un valore troppo ampio con il rischio di individuare falsi minimi nell'energia, ma neanche scegliere un valore troppo ristretto che potrebbe portare ad escludere la soluzione corretta. Il terzo tema che sarebbe utile affrontare, è quello di adottare tecniche per la gestione efficiente della memoria, al fine di superare il limite di 250 frame oltre il quale la GPU utilizzata genera errori di tipo Out of Memory.



# Bibliografia

- [1] Shaowei Liu, David Yifan Yao, Saurabh Gupta, and Shenlong Wang. Visual Sync: Multi-Camera Synchronization via Cross-View Object Motion, December 2025.
- [2] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, and Lei Zhang. Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection, July 2024.
- [3] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment Anything in Images and Videos.
- [4] Ho Kei Cheng, Seoung Wug Oh, Brian Price, Alexander Schwing, and Joon-Young Lee. Tracking Anything with Decoupled Video Segmentation, September 2023.
- [5] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. VGGT: Visual Geometry Grounded Transformer.
- [6] Nikita Karaev, Iurii Makarov, Jianyuan Wang, Natalia Neverova, Andrea Vedaldi, and Christian Rupprecht. CoTracker3: Simpler and Better Point Tracking by Pseudo-Labeling Real Videos, October 2024.
- [7] Vincent Leroy, Yohann Cabon, and Jérôme Revaud. Grounding Image Matching in 3D with MAST3R, June 2024.
- [8] Georg Bieker. A global comparison of the life-cycle greenhouse gas emissions of combustion engine and electric passenger cars.